DATA305 / DATA475 · Machine Learning Methods

Training Deep Networks I

Gradients · Initialization · Activations · Batch Normalization

Lecture 4

Week 6 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda

Four topics · CIFAR-10 running example

01 Vanishing & Exploding Gradients

Sigmoid saturation, gradient norm through layers, chain rule

02 Weight Initialization

Glorot / Xavier and He init — matched to activation function

03 Modern Activation Functions

LeakyReLU, ELU, GELU — avoiding left-side saturation

04 Batch Normalization & Gradient Clipping

Normalizing activations, learnable γ/β, clipnorm vs clipvalue

Dataset: keras.datasets.cifar10 (50 000 train / 10 000 test, 32×32 RGB, 10 classes) · Geron Ch. 11 pp. 357–401

Lecture 3 → Lecture 4

In L3 we mastered the Functional API, multi-output models, and Keras callbacks. Today: why do deep networks struggle to train, and how do we fix it?

Lecture 3 — Covered

- Functional API: Input() → layer chaining → Model()
- Wide & Deep architecture as motivating example
- Multi-output models: per-output losses, loss weights
- Callbacks: EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
- Model persistence: save() / load_model()

Lecture 4 — Today

- Vanishing / exploding gradients — the core training challenge
- Glorot and He initialization — matched to activation function
- Modern activations: LeakyReLU, ELU, GELU
- Batch Normalization: γ/β, train vs. inference mode
- Gradient Clipping: clipnorm vs. clipvalue

Running example: CIFAR-10 MLP — apply each fix one by one, compare val_accuracy and gradient norms.

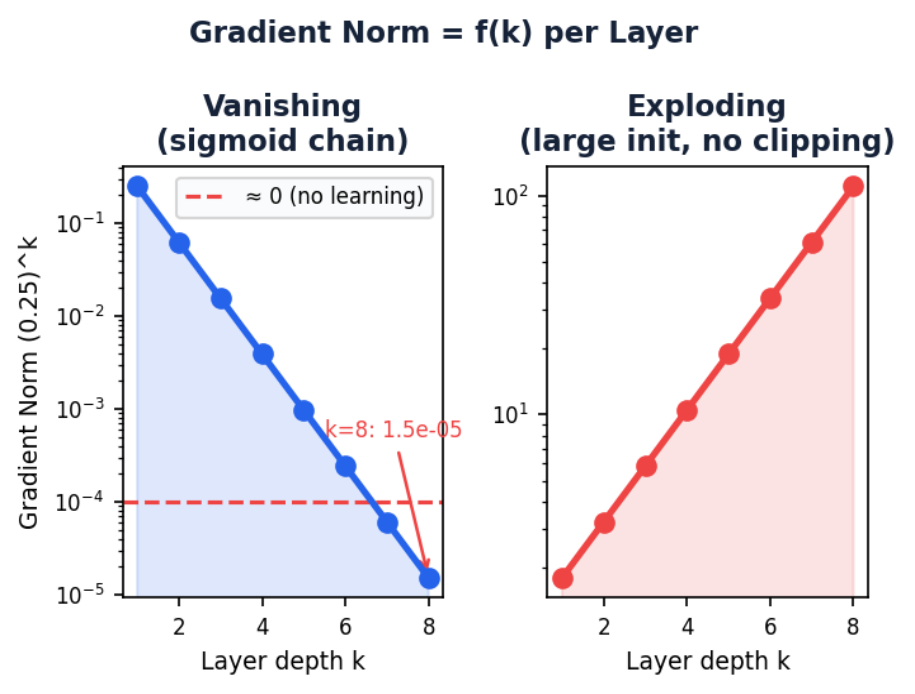
Why Depth Hurts — Vanishing Gradients

The Root Problem

- Backpropagation multiplies gradients layer-by-layer via the chain rule.
- With sigmoid: $\sigma'(z) \leq 0.25$ → activation derivative contributes at most a factor of 0.25 per layer.
- **After 8 layers: $0.25^8 \approx 1.5 \times 10^{-5}$ — effectively zero. No learning signal.**

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a_n} \cdot \prod_i \sigma'(z_i) \cdot \prod_i W_i$$

- Exploding gradients: large init or no clipping → gradient grows exponentially.
- **Symptom: NaN loss. Fix: gradient clipping (covered in Section 04).**



Gradient norm (log scale) per layer — sigmoid-chain vs. unstable init.

Weight Initialization — Glorot and He

Goal: keep activation variance ≈ constant across all layers — no saturation, no explosion.

Naïve Init

e.g. std = 1.0 (naïve large random)

- Activations saturate at ±1 after 1–2 layers
- Gradient → 0 immediately (vanishing)
- Avoid: this is the baseline to beat

Glorot / Xavier

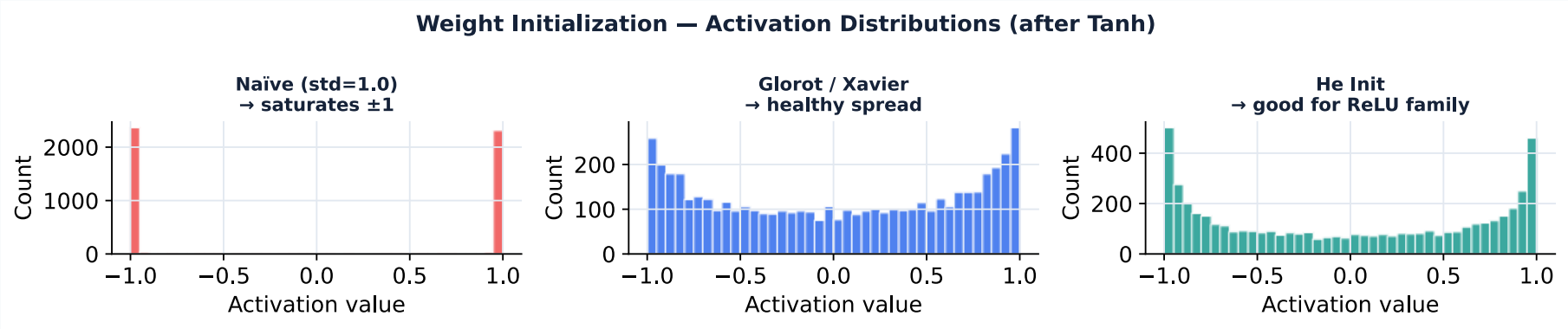
$$std = \sqrt{\frac{2}{f_{in} + f_{out}}}$$

- Designed to keep activation variance stable across layers
- Default in Keras: kernel_initializer='glorot_uniform'
- Best for Tanh and Sigmoid activations

He Initialization

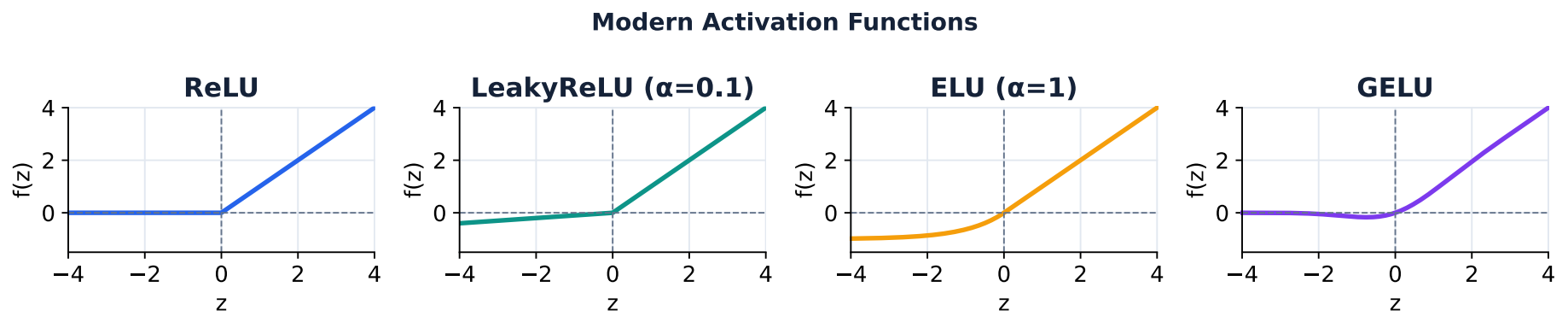
$$std = \sqrt{\frac{2}{f_{in}}}$$

- ReLU zeros ~50% of inputs → need larger variance
- Use: kernel_initializer='he_normal'
- Best for ReLU, LeakyReLU, ELU, GELU



Activation distributions after one Tanh layer — naïve init saturates; Glorot and He preserve spread.

Modern Activation Functions



These functions address or reduce ReLU's left-side inactivity in different ways. ELU and GELU are smooth.

ReLU	LeakyReLU	ELU	GELU
max(0, z)	**z if z≥0; αz if z<0 (α=0.1)**	**z if z≥0; α(e^z−1) if z<0**	**z · Φ(z) (Gaussian CDF)**
Simple and fast. Neurons can become inactive for many updates if stuck in the negative region.	Small negative slope keeps neurons alive. Good general-purpose choice.	Smooth negative region, mean activation closer to 0. Often better accuracy than ReLU.	Common in Transformers. Smooth, probabilistic. Standard in BERT, GPT.

Batch Normalization — The Algorithm

Four Steps — Forward Pass (Training)

1 **Compute mini-batch mean**

$$\mu_B = \frac{1}{m}\sum_i x_i$$

2 **Compute mini-batch variance**

$$\sigma_B^2 = \frac{1}{m}\sum_i (x_i - \mu_B)^2$$

3 **Normalize each activation**

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

4 **Scale and shift (learnable)**

$$y_i = \gamma\,\hat{x}_i + \beta$$

Train vs. Inference

Training mode: Uses current mini-batch μ_B and σ²_B. Each batch slightly different.

Inference mode: Uses running mean/variance accumulated during training. Fixed, deterministic.

γ and β: Learnable parameters — allow the network to undo normalization when needed.

ε (epsilon): Small constant (~1e-5) for numerical stability in the denominator.

Key Rule: Dense → BN → Activation

```
Dense(units, kernel_initializer='he_normal')  →
BatchNormalization()  ←  normalizes here
Activation('relu')
```

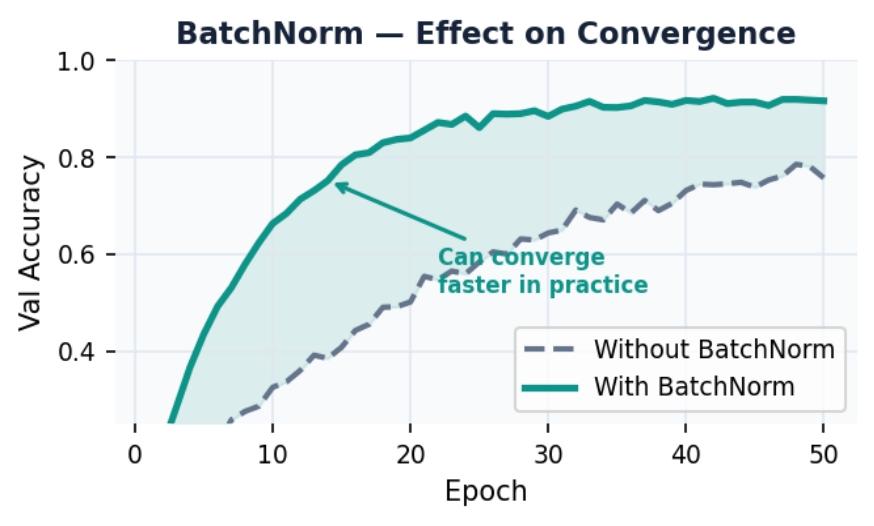
Batch Normalization — Effect and Keras Usage

Keras Usage

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(32,32,3)),
    Flatten(),
    Dense(256, kernel_initializer='he_normal'),
    BatchNormalization(),
    Activation('relu'),
    Dense(128, kernel_initializer='he_normal'),
    BatchNormalization(),
    Activation('relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=[tf.keras.metrics.
    SparseCategoricalAccuracy(name='accuracy')]
)
```

- Note: Do NOT apply BatchNorm at the output layer.
- During inference Keras automatically uses running stats — no extra code needed.



BN can improve convergence speed and stability — especially in deeper networks. Benefits depend on architecture and training length.

Faster convergence: Can improve gradient flow and stability. In shallow MLPs, gains may be small or absent.

Training stability: Can make training less sensitive to learning rate choice.

Mild regularization: Acts as a weak regularizer, modestly reducing overfitting.

Gradient Clipping

Two Strategies

Clip by Norm

clipnorm=1.0

Rescales the entire gradient vector so its global L2 norm ≤ threshold. Gradient direction preserved. Recommended default for most architectures.

Clip by Value

clipvalue=0.5

Clips each gradient element independently — changes direction. Less common in modern feed-forward setups.

```
opt = tf.keras.optimizers.Adam(
    learning_rate=1e-3,
    clipnorm=1.0   # clip by global L2 norm
)
model.compile(optimizer=opt, ...)
```

Practical Heuristics — Quick Reference

Activation	Recommended Init	When to use
ReLU	He Normal	Default for dense / conv layers
LeakyReLU	He Normal	When dead neurons are a concern
ELU	He Normal	Smooth outputs, deep MLPs
GELU	He Normal	Transformers, modern architectures
Tanh / Sigmoid	Glorot Uniform	Gates, RNNs, output layers

Add clipnorm=1.0 to any optimizer as a safe default for deep networks.

Putting It Together — Colab Code

python · CIFAR-10: He Init + BatchNorm + clipnorm

```python
import tensorflow as tf

class GradNormLogger(tf.keras.callbacks.Callback):
    def __init__(self, x_s, y_s):
        super().__init__()
        self.xs = tf.constant(x_s, dtype=tf.float32)
        self.ys = tf.constant(y_s, dtype=tf.int32)
        self.loss_fn = tf.keras.losses.SparseCategoricalCrossentropy()
    def on_epoch_end(self, epoch, logs=None):
        with tf.GradientTape() as tape:
            loss = self.loss_fn(self.ys, self.model(self.xs, training=True))
        grads = tape.gradient(loss, self.model.trainable_variables)
        norms = [tf.linalg.norm(g) for g in grads if g is not None]
        logs['mean_grad_norm'] = float(tf.reduce_mean(norms))

(x_tr, y_tr), _ = tf.keras.datasets.cifar10.load_data()
x_tr = x_tr.astype('float32') / 255.0
y_tr = y_tr.ravel().astype('int32')    # must be 1-D int32

model = tf.keras.Sequential([
    tf.keras.Input(shape=(32,32,3)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, kernel_initializer='he_normal'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation('relu'),
    tf.keras.layers.Dense(128, kernel_initializer='he_normal'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation('relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])
opt = tf.keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)
model.compile(optimizer=opt, loss='sparse_categorical_crossentropy',
    metrics=[tf.keras.metrics.SparseCategoricalAccuracy(name='accuracy')])
x_s = x_tr[:64]; y_s = y_tr[:64]
model.fit(x_tr, y_tr, epochs=30, validation_split=0.1, batch_size=256,
    callbacks=[GradNormLogger(x_s, y_s)])
```

GradNormLogger

Uses GradientTape to compute actual gradient norms (not weight norms). Logs mean_grad_norm per epoch.

he_normal

He init: std = √(2/fan_in). Correct for ReLU — compensates for zeroing half of inputs.

BatchNormalization()

After Dense, before Activation. Normalizes activations per mini-batch. Learnable γ and β.

clipnorm=1.0

Global gradient clipping — rescales gradient vector if L2 norm exceeds 1. Direction preserved.

QUICK QUIZ

You train a 10-layer MLP with ReLU activations on CIFAR-10. After epoch 1, val_accuracy stays at 10% (random chance). You check mean_grad_norm and see ≈ 0.00001 logged by GradNormLogger. Which combination is MOST likely to help?

A Switch to Sigmoid + Glorot init — Sigmoid is more stable for deep nets.

B Switch to He init + add BatchNormalization + keep ReLU.

C Add more layers and increase learning rate to 0.1.

D Use clipvalue=0.5 — the gradient norm indicates explosion, not vanishing.

Hint: what does mean_grad_norm ≈ 0.00001 tell you about the gradient flow?

Run the Notebook — Colab Walkthrough

1

Section 1–2: Vanishing gradients + Init

Run cells 1.1–2.2. Observe gradient norms on the 8-layer sigmoid MLP. Compare Glorot vs He init.

2

Section 3: Activation function plots

Run cells 3.1–3.2. Plot ReLU, LeakyReLU, ELU, GELU. Compare val_accuracy across activations.

3

Section 4–5: BatchNorm + Gradient Clipping

Run cells 4.1–5.1. Compare no-BN vs BN convergence. Note: benefits vary with architecture depth.

4

Section 6–7: Full stack + Experiments

Enable GPU (Runtime → Change runtime type → T4). Run Section 6 (He + BN + clipnorm=1.0) and Section 7 experiments. Record results.

Lecture 4 Summary

Key Takeaways

Vanishing Gradients

$\sigma'(z) \leq 0.25 \rightarrow$ gradient shrinks per layer. After 8 layers $\approx 10^{-5}$. Fix: better init + activations.

He / Glorot Init

He (ReLU family): $\text{std}=\sqrt{2/\text{fan}_{in}}$. Glorot (Tanh/Sigmoid): $\text{std}=\sqrt{2/(\text{fan}_{in}+\text{fan}_{out})}$.

Modern Activations

ReLU → LeakyReLU → ELU → GELU. Reduce left-side inactivity in different ways. GELU: standard in Transformers.

Batch Normalization

Dense → BN → Activation. Normalize per-batch with learnable γ, β. Can improve optimization stability.

Gradient Clipping

clipnorm=1.0 on optimizer. Rescales gradient so L2 norm ≤ 1. Direction preserved.

Practical Stack

he_normal + BatchNorm + ReLU/LeakyReLU + clipnorm=1.0 — a strong default for deep MLPs.

Why Sigmoid Gradients Vanish — Detailed Derivation

1. **Chain rule multiplies gradients layer-by-layer**

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \frac{\partial a_{n-1}}{\partial a_{n-2}} \cdots \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial W_1}$$

2. **For each hidden layer**

$$a_\ell = \sigma(z_\ell), \qquad z_\ell = W_\ell a_{\ell-1} + b_\ell$$

$$\frac{\partial a_\ell}{\partial a_{\ell-1}} = \frac{\partial a_\ell}{\partial z_\ell} \cdot \frac{\partial z_\ell}{\partial a_{\ell-1}} = \sigma'(z_\ell) W_\ell$$

so, up to scale,

$$\frac{\partial L}{\partial W_1} \propto \frac{\partial L}{\partial a_n} \cdot \prod_{\ell=2}^{n} \sigma'(z_\ell) \cdot \prod_{\ell=2}^{n} W_\ell$$

3. **With sigmoid, each layer shrinks the signal**

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) \leq 0.25$$

Therefore after k sigmoid layers:

$$|g_k| \leq 0.25^k |g_0|$$

After 8 layers:

$$0.25^8 \approx 1.5 \times 10^{-5}$$

Effectively zero: almost no useful learning signal reaches early layers.

Why Xavier Init Uses $2/(fan_{in} + fan_{out})$

1. **Goal**

   Keep variance roughly stable in both directions:

   - forward pass — activations should not explode or vanish
   - backward pass — gradients should also stay well-scaled

2. **Forward-pass intuition**

   For one linear layer:

$$z = Wx$$

   Assuming inputs and weights are approximately zero-mean and independent:

$$Var(z) \approx fan_{in}\, Var(w)\, Var(x)$$

   So, to keep forward activations stable:

$$Var(w) \approx \frac{1}{fan_{in}}$$

3. **Backward-pass intuition**

   Backpropagated gradients pass through W^T, giving a similar condition:

$$Var(w) \approx \frac{1}{fan_{out}}$$

4. **Xavier / Glorot compromise**

   Balance the forward and backward constraints by averaging them:

$$Var(w) \approx \frac{2}{fan_in + fan_out}$$

Why He Init Uses $2/fan_{in}$ — Intuition

1. **Start from one pre-activation in a dense layer**

$$z = \Sigma_{i=1}{}^{\wedge} fan_{in}\, w_i\, x_i$$

Assume $E[x_i]=0$, $Var(x_i)=v$, $E[w_i]=0$, $Var(w_i)=\sigma_w^2$.

$$Var(z) \approx fan_{in} \cdot \sigma_w^2 \cdot v$$

2. **For ReLU, about half of the symmetric input mass is zeroed**

$$Var(ReLU(z)) \approx ½\, Var(z)$$

So to keep activation variance roughly constant across layers, require $Var(a) \approx v$.

3. **Set the output variance equal to the input variance**

$$½ \cdot fan_{in} \cdot \sigma_w^2 \cdot v \approx v$$

$$\sigma_w^2 \approx 2 / fan_{in} \quad \Rightarrow \quad std(w) = \sqrt{(2 / fan_{in})}$$

Therefore He initialization uses the factor 2/fan_in for ReLU-family activations.

Why BN Uses Running Statistics at Inference

1. **During training, BatchNorm uses the current mini-batch**

$$\mu_B = (1/m)\,\Sigma_i\, x_i \qquad \sigma_B^2 = (1/m)\,\Sigma_i\,(x_i - \mu_B)^2$$

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}$$

2. **At inference, the batch may be tiny or arbitrary**

   If we reused the current batch statistics, the prediction for the same sample could change depending on the other samples in the batch.

3. **So BN keeps running estimates during training**

$$\mu_running \leftarrow \rho\,\mu_running + (1-\rho)\,\mu_B$$

$$\sigma_running^2 \leftarrow \rho\,\sigma_running^2 + (1-\rho)\,\sigma_B^2$$

   Inference then uses fixed stored statistics

$$\hat{x} = (x - \mu_running) / \sqrt{(\sigma_running^2 + \varepsilon)}$$

Result: deterministic predictions that do not depend on batch composition.